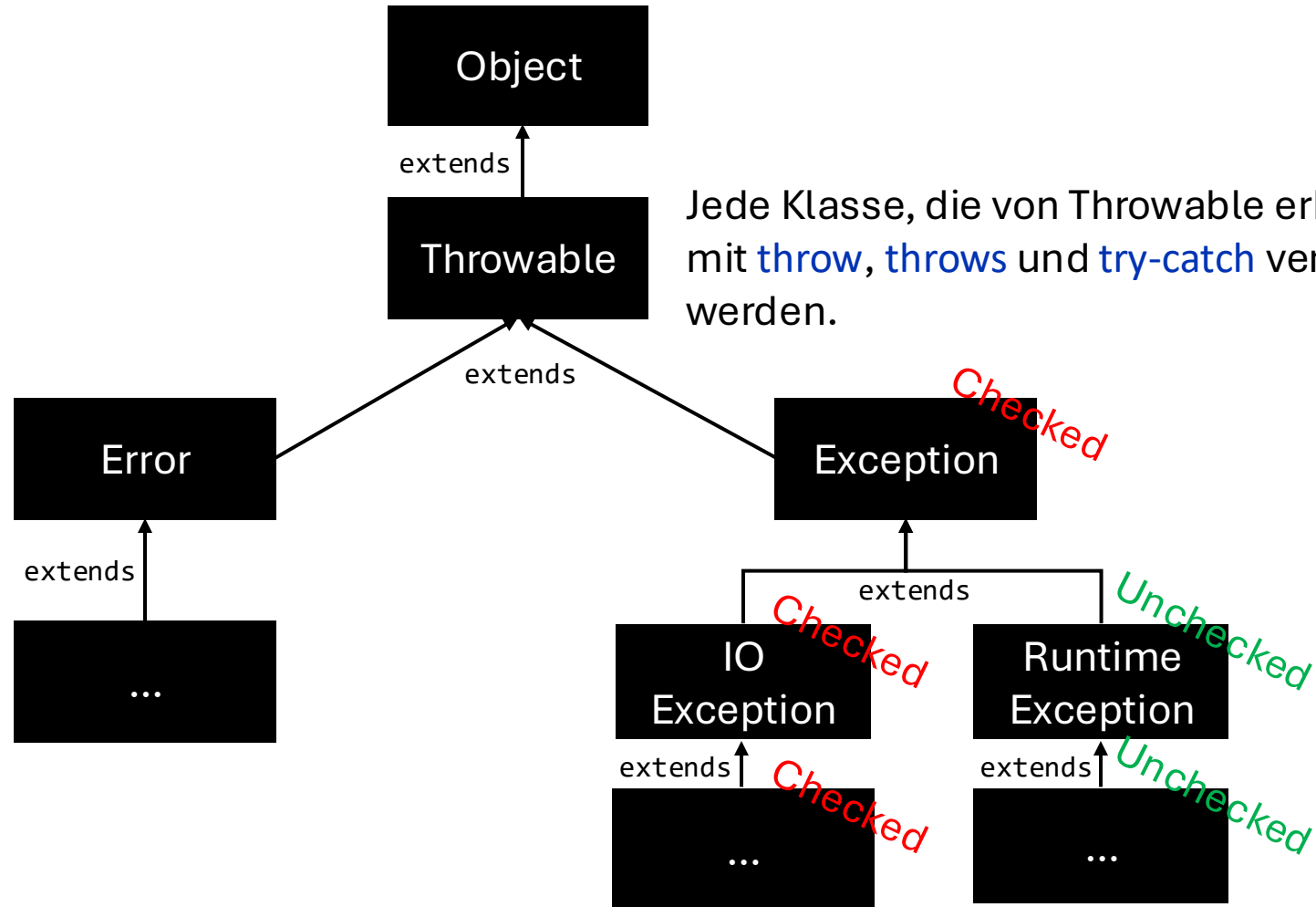




Exceptions, Optional und Null Safety

Exceptions in Java



Jede Klasse, die von Throwable erbt, kann mit **throw**, **throws** und **try-catch** verwendet werden.

"Ein Error weist auf ernsthafte Probleme hin, die eine sinnvolle Anwendung nicht abfangen sollte. Error finden unter abnormalen Bedingungen statt, die niemals auftreten sollten." Beispiel: Hardwarefehler.

Checked Exception

Eine Checked Exception muss **vorm Kompilieren behandelt** werden durch:

- try-catch-Block
- Methodensignatur (leitet das "Problem" an den Aufrufer weiter).

Beispiele:

Exception und alle seine Ableitungen wie z. B. **IOException** oder **SQLException** (außer RuntimeException).

Unchecked Exception

Eine Unchecked Exception muss **nicht vorm Kompilieren behandelt** werden. Sie tritt erst zur **Laufzeit** auf.

Unchecked Exception in der Methodensignatur ist **optional**.
Leitet das "Problem" transparent weiter an den Aufrufer.

Beispiele:

RuntimeException und alle seine Ableitungen wie z. B.
NoSuchElementException, **NullPointerException** oder
ArrayIndexOutOfBoundsException.

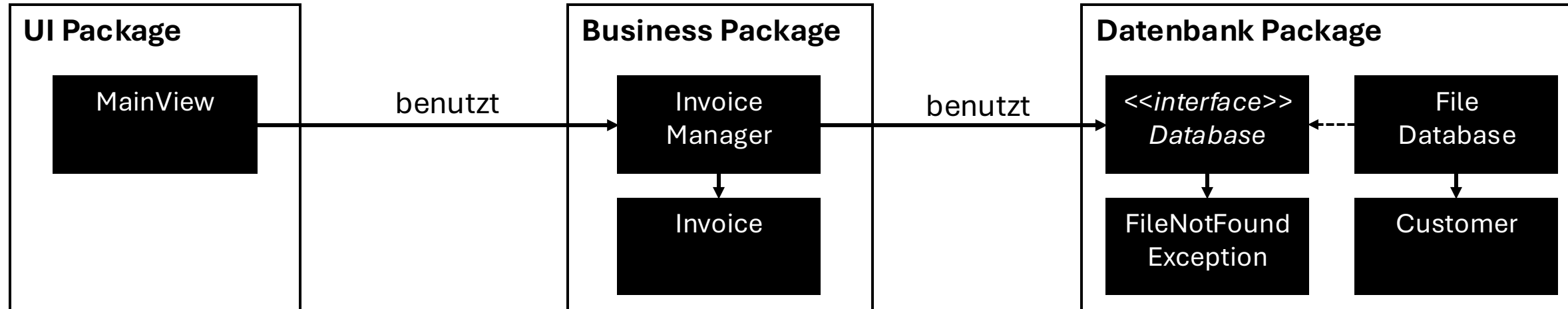
Beispiel: Invoice App

Legende:

Package

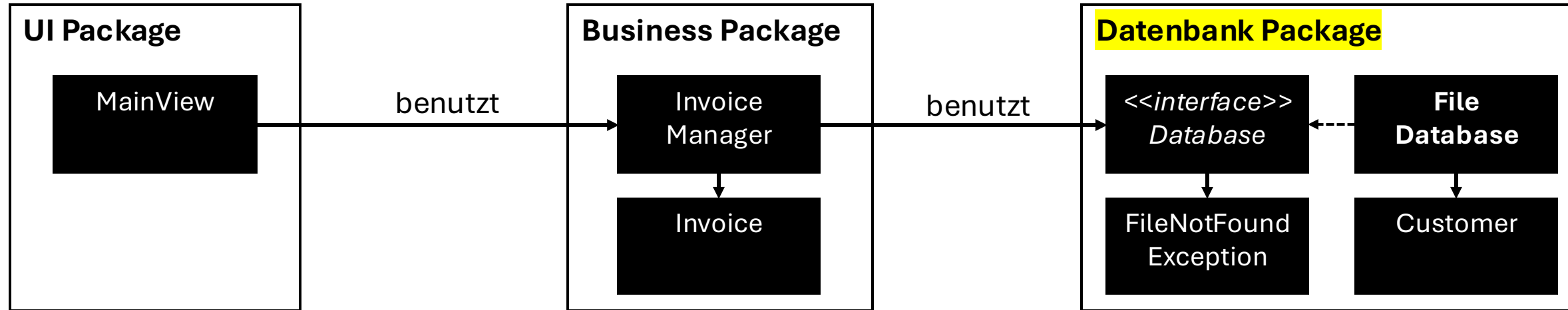
Klasse

Interface



Package
Klasse
Interface

Beispiel: Checked Exception 1/3



findCustomer(...) wirft eine Checked **FileNotFoundException**, die von **IOException** erbt.

```

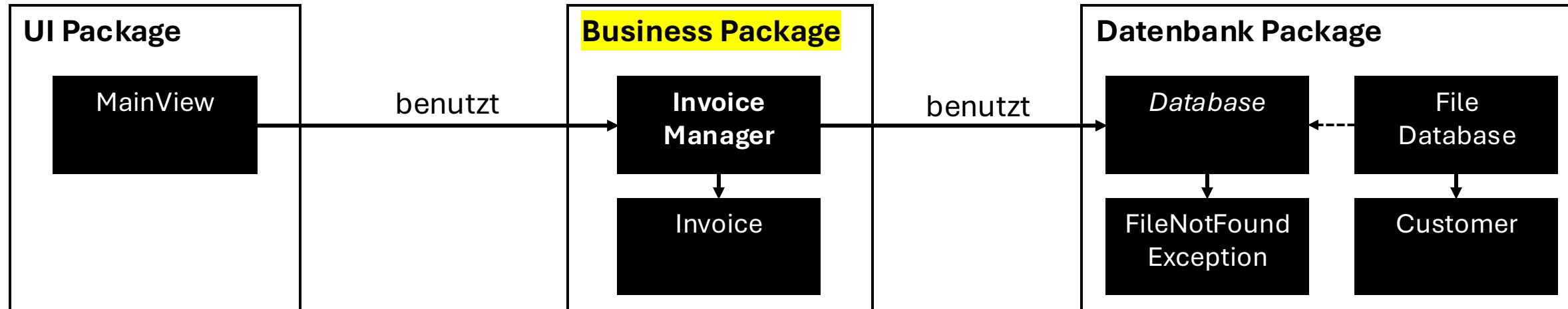
public class FileNotFoundException extends IOException {
    public FileNotFoundException(String message) {
        super(message);
    }
}
    
```

```

public class FileDatabase implements Database {
    @Override
    public Collection<Customer> findCustomer(String name) throws FileNotFoundException {
        try {
            String fileContent = Files.readString(Path.of(CUSTOMERS_DB_FILE)); // wirft IOException
            var customers = parseByCustomer(name, fileContent);
            return Collections.unmodifiableCollection(customers);
        } catch (IOException e) {
            throw new FileNotFoundException(CUSTOMERS_DB_FILE); // Exception-Umwandlung
        }
    }
}
    
```

Package
Klasse
Interface

Beispiel: Checked Exception 2/3



```
public class InvoiceManager {
```

```
    private final Database database;
```

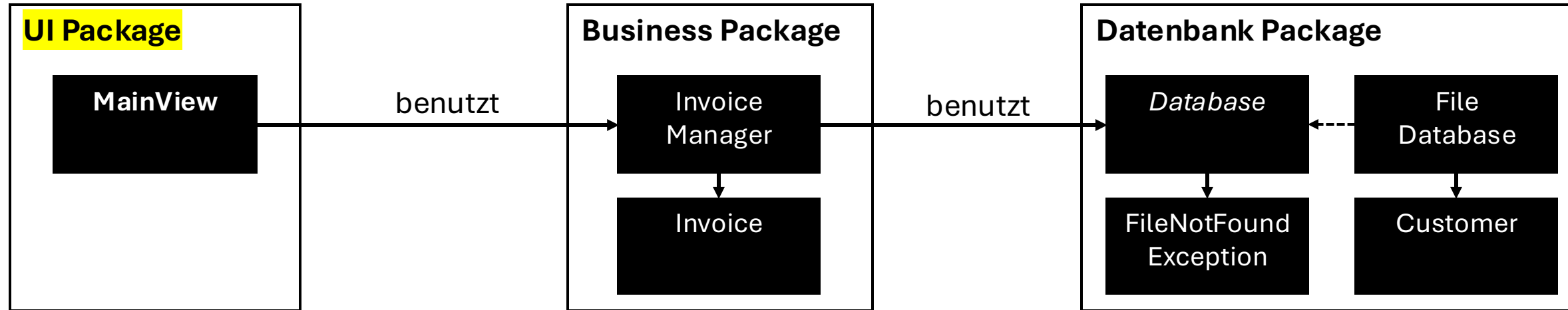
```
    public InvoiceManager(Database database) { this.database = database; }
```

```
    public Invoice calculateByCustomer(String name) throws FileNotFoundException {
        var customer = database.findCustomer(name); // findCustomer wirft Checked Exception
        // do calculations with customer and fill result...
        return new Invoice(42.50);
    }
}
```

InvoiceManager muss sich mit FileNotFoundException beschäftigen, obwohl die Klasse nichts mit Dateien zu tun hat.

Package
Klasse
Interface

Beispiel: Checked Exception 3/3



```

public class MainView {
    private final InvoiceManager invoiceManager; // Dependency Injection: main(String args) { new MainView(new InvoiceManager(new FileDatabase)); }
    public MainView(InvoiceManager invoiceManager) { this.invoiceManager = invoiceManager; }
    public void showInvoiceByCustomer(String name) {
        try {
            var invoice = invoiceManager.calculateByCustomer(name); // wirft FileNotFoundException
            System.out.println(invoice);
        } catch (FileNotFoundException e) {
            System.err.println("Error: Database is currently unavailable.");
        }
    }
}
    
```

MainView **muss** sich mit **FileNotFoundException** beschäftigen, obwohl die Klasse nichts mit Dateien zu tun hat.

Fazit: Checked Exception

Bei Exceptions gibt es häufig lokal keine sinnvolle Reaktion für einen `try-catch`-Block. Stattdessen wird die Exception zur Methodensignatur hinzugefügt und immer weiter durch das Softwaresystem durchgereicht.

Problem: Eine Checked Exception taucht somit an vielen Stellen im Code auf. Dadurch werden die Software-Prinzipien Kapselung und Minimierung von Abhängigkeiten verletzt.

In der Praxis haben sich daher größtenteils Unchecked (Runtime) Exceptions durchgesetzt*.

* Für kritische Systeme können Checked Exception nach wie vor sinnvoll sein.
Beispiele: Raketen, Atomkraftwerk, selbstfahrendes Auto, Banking App

Bad Software Design?

Das Beispiel ist nicht optimal und "flawed by design". 

Die Schnittstelle zur Datenbank ist wie folgt:

```
public interface Database {  
    Collection<Customer> findCustomer(String name) throws FileNotFoundException;  
}
```

Was passiert, wenn weitere Exceptions hinzugefügt werden sollen? 

Option 1: Mehr Checked Exception

```
public interface Database {  
    Collection<Customer> findCustomer(String name) throws FileNotFoundException, FileCorruptedException,  
                                                             FileNotAccessableException;  
}
```

Weitere File-Exception nach `throws` hinzufügen.

 Dies hat den Nachteil, dass sich bei jeder Änderung der Methodensignatur die Database-Schnittstelle ändert und alle Klassen, die diese Schnittstelle **direkt oder indirekt** verwenden, angepasst werden müssen.

 **Merke:** Schnittstellen (Interfaces) sollten sich möglichst selten ändern. Am besten nie, das ist allerdings unrealistisch. Es sollte aber das Design-Ziel sein!

Einschub: Warum eigene Exceptions?

Java enthält bereits **vorgefertigte Exceptions**, die frei verwendet werden können wie z. B. `IllegalArgumentException`, `IllegalStateException`, `IOException`, `SQLException`, `ParseException`, `MalformedURLException`, und viele weitere.

Für den Compiler macht es keinen Unterschied, ob eine `IOException` oder eine `SQLException` geworfen wird. Hier spielt her die Semantik (Bedeutung) eine Rolle: die Exception soll uns Entwicklern bereits durch ihren Namen einen Hinweis auf den Fehler geben.

Der **Vorteil** von **eigenen** Exceptions:

Man kann sie gezielt mit `catch` abfangen und so auf eine bestimmte Exception reagieren. Wenn ausschließlich `IOException` geworfen werden, dann kann man sie im Code nicht voneinander unterscheiden.

Außerdem wird ein Code **verständlicher**, wenn eine Methode z. B. eine `FileNotFoundException` wirft statt eine generische `IOException`.

Option 2: Generische Checked Exception

```
public interface Database {  
    Collection<Customer> findCustomer(String name) throws IOException;  
}
```

Generische IOException wäre für das Interface sinnvoller. Denn dadurch sind beliebig weitere IOException möglich (durch Vererbung von IOException) und die Methodensignatur **bleibt gleich**.

Das heißt, es können **beliebige** Exceptions von der Methode findCustomer geworfen werden, solange sie **von IOException erben**, z. B. FileNotFoundException, FileCorruptedException, und FileNotAccessableException.

Außerdem würde eine generische IOException (statt eine FileNotFoundException) **semantisch weitere Implementationen** für das Database-Interface unterstützen, z. B. falls zukünftig die FileDatabase durch eine SQLiteDatabase ersetzt wird. Durch Vererbung von IOException wäre z. B. auch SQLException möglich.

Option 3: Unchecked Exception

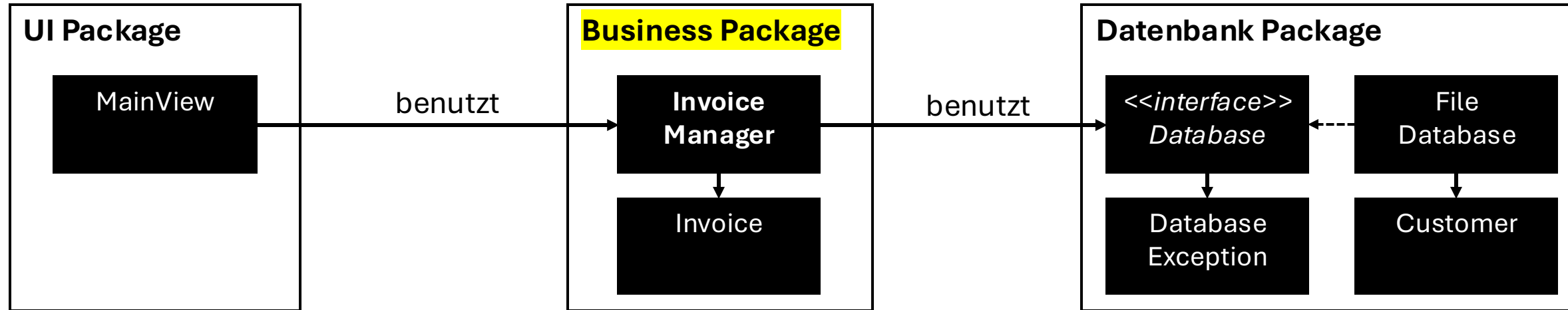
```
public class DatabaseException extends RuntimeException { // Unchecked Exception, da RuntimeException
    public DatabaseException(String message) { super(message); }
}

public interface Database {
    Collection<Customer> findCustomer(String name) throws DatabaseException; // optionale throws-Angabe
}

public class FileDatabase implements Database {
    @Override
    public Collection<Customer> findCustomer(String name) {
        try {
            String fileContent = Files.readString(Path.of(CUSTOMERS_DB_FILE)); // Files.readString wirft eine IOException
            var customers = parseByCustomer(name, fileContent);
            return Collections.unmodifiableCollection(customers);
        } catch (IOException e) {
            throw new DatabaseException("File not found: " + CUSTOMERS_DB_FILE); // Umwandlung von Checked IOException
                                                    // in Unchecked DatabaseException
        }
    }
}
```

Alle Aufrufer der Methode `findCustomer(...)` müssen **keine** Exception mehr behandeln. Die Exception kann so lange ignoriert werden, bis sie an einer sinnvollen Stelle behandelt (`try-catch`) wird, z. B. im MainView, der den Fehler anzeigt.

Beispiel: Unchecked Exception 1/2



InvoiceManager kann die DatabaseException ignorieren und hat dadurch auch keine Referenz bzw. Abhängigkeit zur DatabaseException.

```

public class InvoiceManager {

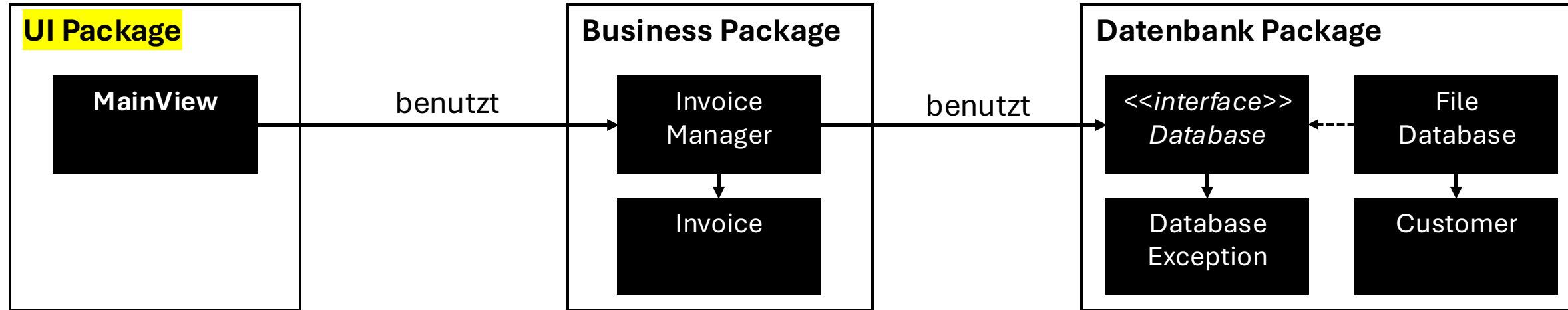
    private final Database database;

    public InvoiceManager(Database database) { this.database = database; }

    public Invoice calculateByCustomer(String name) {
        var customer = database.findCustomer(name); // mögliche DatabaseException wird ignoriert
        // do calculations with customer and fill result...
        return new Invoice(42.50);
    }
}
    
```

Package
Klasse
Interface

Beispiel: Unchecked Exception 2/2



```

public class MainView {

    public void showInvoiceByCustomer(String name) {
        try {
            var invoiceManager = new InvoiceManager(new FileDatabase());
            var invoice = invoiceManager.calculateByCustomer(name);
            System.out.println(invoice);
        } catch (DatabaseException e) {
            System.err.println("Error: Database is currently unavailable.");
        } catch (Exception e) {
            System.err.println(e.getMessage());
        }
    }
}
    
```

MainView fängt mit `catch(Exception e)` **alle** möglichen Exceptions ab, ansonsten würde es eventuell zum Programmabsturz kommen.

Optional möglich: Die `DatabaseException` explizit abfangen, um (für den User) angepasste Fehler auszugeben.

Checked oder Unchecked Exception?

Java basiert auf der *Java Virtual Machine* (JVM) und ist eine der wenigen Programmiersprachen, die Checked Exceptions unterstützt. Weitere Sprachen mit Checked Exceptions sind Kotlin (JVM), Scala (JVM) und Objective-C.

Checked Exceptions haben sich kaum in anderen Programmiersprachen etabliert, da oft die Flexibilität von Unchecked Exceptions bevorzugt werden.

Globales Exception Handling?

Frage: Warum nicht alle Exceptions global "ganz oben" in der main-Methode abfangen?

```
public static void main(String[] args) {  
    try {  
        // start application  
        new MainView().show();  
        // run app ...  
    } catch (Exception e) {  
        System.err.println(e.getMessage());  
    }  
}
```

Antwort: In der Regel möchte man eine Exception möglichst lokal abfangen und behandeln (Fail Fast Prinzip), z. B. weil der Fehler im Dialog-Fenster angezeigt werden soll und nicht im Hauptfenster. Es ist zudem oft sinnvoller, sofern möglich, Exceptions lokal zu behandeln, damit sie nicht überall im Softwaresystem auftauchen.

Darüber hinaus sind Exceptions langsam im Vergleich zu if-else-Abfragen, da beim Werfen einer Exception jedes Mal ein Call-Stack (Historie der Methodenaufrufe) erzeugt wird.

Fail Fast Principle



Je später Fehler behandelt werden, **desto schwieriger** lässt sich die **Fehlerursache** im Nachhinein ermitteln.

Aus diesem Grund besagt das Fail Fast Principle:

- Fehler so früh wie möglich erkennen.
- Fehler so früh wie möglich zurückmelden.
- Fehler möglichst lokal behandeln und nicht weitergeben.

Ziel: Einfache Diagnose und Debugging.

Fail Fast Principle: Beispiel (Preconditions)

Prüfe **zu Beginn** einer **Methode** oder eines **Konstruktors**, ob die Parameter **valide** sind.

Beispiel:

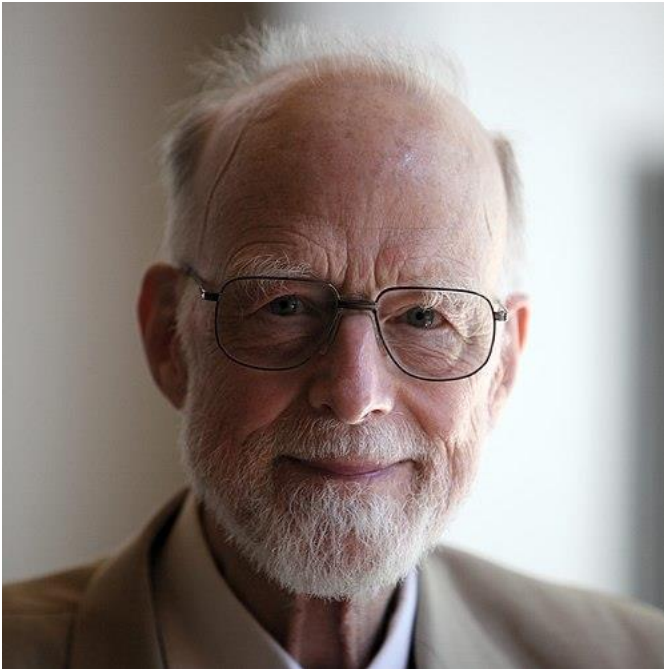
```
Optional<Invoice> calculateByCustomer(String name) {  
    // 1. do validity checks first (preconditions).  
    if(name == null || name.isBlank()) {  
        throw new IllegalArgumentException(name); // or use Optional<T> instead of Exception (see upcoming slides).  
    }  
  
    // 2. then do the actual operation with valid parameters.  
    // query customer by name...  
}
```

Die Prüfung am Anfang der Methode nennt man auch **Precondition**: die Methode prüft zunächst alle Parameter auf Gültigkeit. Ist ein Parameter ungültig, wird die Methode sofort mit einer Exception abgebrochen, da eine Vorbedingung verletzt wurde und die Methode nicht korrekt ausgeführt werden könnte.

null



Erfinder des Null



Tony Hoare

„I call it my **billion-dollar mistake**. It was the invention of the null reference in 1965. At that time, I was designing the first comprehensive type system for references in an object oriented language (ALGOL W).

My goal was to ensure that all use of references should be absolutely safe, with checking performed automatically by the compiler. But **I couldn't resist the temptation to put in a null reference, simply because it was so easy to implement.**

This has led to innumerable errors, vulnerabilities, and system crashes, which have probably caused a billion dollars of pain and damage in the last forty years.”

Null Problemo



`null` sollte möglichst vermieden werden, da sonst überall im Code `null`-Checks erforderlich sind:

```
class InvoiceManager {  
  
    Invoice calculateByCustomer(String name) {  
        var customer = database.findCustomer(name);  
        if (customer != null) {  
            // do calculations with customer and fill result...  
            return new Invoice(42.50);  
        } else {  
            return null; // Der Aurufer von calculateByCustomer muss wieder einen null-Check für die Invoice machen... 🤪  
        }  
    }  
}
```

Primitive statt null

Primitive (`byte`, `short`, `int`, `long`, `float`, `double`, `boolean`, `char`) sind, soweit möglich, vorzuziehen, da diese niemals `null` sein können. 👍

`int number = null;` // Nicht kompilierbar. Nur Objekte können null sein.

Objekte können dagegen `null` sein, wie z. B. bei den Wrapper-Klassen Integer, Double, Character usw.

```
class MathUtil {  
    static double square(double number) { // Rückgabewert und Parameter kann niemals null sein, kein null-Check notwendig!  
        return number * number;  
    }  
  
    static Double square(Double number) { // Rückgabewert und Parameter kann null sein, null-Check notwendig!  
        return number * number;  
    }  
  
    static void main(String[] args) {  
        System.out.println(square(null)); // Der Compiler wählt hier die Double-Methode aus, weil nur diese Methode null-Werte akzeptiert.  
    }  
}
```


Exception statt null

Eine Möglichkeit, `null` zu vermeiden, wäre eine `RuntimeException` zu werfen.

```
interface Database {  
    Customer findCustomer(String name) throws DatabaseException; // DatabaseException erbt von RuntimeException  
}  
  
class InvoiceManager {  
    public Invoice calculateByCustomer(String name) {  
        var customer = database.findCustomer(name); // wirft eine RuntimeException.  
        // do calculations with customer and fill result...  
        return new Invoice(42.50);  
    }  
}
```

Vorteil: Die `RuntimeException` kann ignoriert werden.

Nachteil: Die Exception ist "versteckt". Sie ist häufig nur im JavaDoc dokumentiert und kann daher übersehen werden. Das kann dazu führen, dass ein `try-catch` vergessen wird (lokal oder global), und die App zur Laufzeit abstürzen kann. Das sollte nicht passieren!

Aus der Übung: Average Exception

Schreibe eine Methode `calculateAverage`. Diese Methode akzeptiert eine `Collection` von Ganzzahlen als Parameter und gibt den Durchschnitt aller enthaltenen Zahlen zurück.

```
static double calculateAverage(Collection<Integer> numbers)
```

Falls der Parameter leer ist, soll die Methode eine `IllegalArgumentException` werfen.

Rufe die Methode jeweils für beide Fälle in einer `main`-Methode auf und gebe das Resultat in der Konsole so aus, dass das Programm nicht abstürzt.

Live Coding Beispiel

```
public class AverageCalculator {  
  
    public static double calculateAverage(Collection<Integer> numbers) {  
        // Was machen wir hier?  
    }  
  
    public static void main(String[] args) {  
        List<Integer> list1 = List.of(1, 2, 3, 4, 5);  
        System.out.println("Berechne Durchschnitt von list1...");  
    }  
}
```

```
public class AverageCalculator {
```

```
    public static double calculateAverage(Collection<Integer> numbers) {
```

```
        // "Happy Path" - funktioniert nur, wenn 'numbers' nicht leer ist
```

```
        return numbers.stream()
```

```
            .mapToInt(n -> n) // mapToInt(n -> n) wandelt den Stream um Diese Methode konvertiert den  
                               Stream<Integer> (Objekte) in einen IntStream (primitive int-Werte). Der Ausdruck  
                               n -> n führt dabei das sogenannte Unboxing durch (wandelt das  
                               Integer-Objekt in einen int-Wert um).
```

```
            .average() // nur auf primitiven Streams
```

```
            .getAsDouble(); // Achtung: Wirft Exception, wenn leer!
```

```
        // .getAsDouble() holt den rohen double-Wert aus einem OptionalDouble-  
        Container heraus. Stellen euch die .average()-Methode so vor: Sie  
        versucht, einen Durchschnitt zu berechnen. Das Ergebnis verpackt sie in  
        eine Box (ein OptionalDouble), die entweder den Wert enthält oder leer ist.
```

```
    }
```

```
    public static void main(String[] args) {
```

```
        List<Integer> list1 = List.of(1, 2, 3, 4, 5);
```

```
        System.out.println("Berechne Durchschnitt von list1...");
```

```
        double avg1 = calculateAverage(list1);
```

```
        System.out.println("Ergebnis: " + avg1); // Ergibt 3.0
```

```
    }
```

```
public class AverageCalculator {
```

```
    public static double calculateAverage(Collection<Integer> numbers) {  
        // "Happy Path" - funktioniert nur, wenn 'numbers' nicht leer ist  
        return numbers.stream()  
            .mapToInt(n -> n)  
            .average()  
            .getAsDouble(); // Achtung: Wirft Exception, wenn leer!  
    }
```

```
    public static void main(String[] args) {
```

```
        // List 1
```

```
        List<Integer> list2 = List.of();
```

```
        System.out.println("\nBerechne Durchschnitt von list2...");
```

```
//Wir wissen schon, dass es einen Fehler werfen wird und anstatt das Programm abstürzen zu lassen verwenden wir try  
und catch
```

```
    try {
```

```
        double avg2 = calculateAverage(list2);
```

```
        System.out.println("Ergebnis: " + avg2);
```

```
    } catch (Exception e) {
```

```
        System.err.println("Mein Fehler: " + e.getClass().getName() + ": " + e.getMessage());
```

```
    }
```

```

public class AverageCalculator {

    public static double calculateAverage(Collection<Integer> numbers) {
        // Schritt 1: Precondition / Fail Fast
        if (numbers == null || numbers.isEmpty()) {
            throw new IllegalArgumentException("Collection darf nicht null oder leer sein.");
        }
        // Schritt 2: "Happy Path" (jetzt sicher)
        // Standard Happy Path nicht geändert
    }

    public static void main(String[] args) {
        // nur catch geändert
    } catch (IllegalArgumentException e) { // Fangen wir die spezifische Exception
        System.err.println("Kontrollierter Fehler: " + e.getMessage());
    }
}

```

Optional<T>



Optional<T> ist eine Wrapper-Klasse: Ein Optional<T>-Objekt enthält ein anderes Objekt. Im Grunde funktioniert ein Optional<T> wie eine Collection<T> mit maximal einem Element (Optional<T> ist allerdings nicht Iterable).

Mit Optional<T> lässt sich in der Programmiersprache ausdrücken, dass ein Objekt eventuell nicht vorhanden ist (anstatt **null** zu verwenden).

Durch die `ifPresent`- und `orElse`-Methoden in Optional<T> kann das enthaltene Objekt herausgeholt und gleichzeitig eine Alternative codiert werden, falls das Objekt nicht vorhanden (**null**) ist. Dadurch lassen sich if-else-Abfragen vermeiden. 🤖

Aus der Übung: Average Optional

Schreibe eine Methode `calculateAverage`. Diese Methode akzeptiert eine Collection von Ganzzahlen als Parameter und gibt den Durchschnitt, sofern möglich, aller enthaltenen Zahlen zurück.

```
static Optional<Double> calculateAverage(Collection<Integer> numbers)
```

Falls die übergebende Collection leer ist, soll stattdessen ein leeres Optional zurückgegeben werden.

Rufe die Methode für beide Fälle (leer und nicht leer) in einer main-Methode auf und gebe beide Ergebnisse in der Konsole aus.

Optional<T> erzeugen

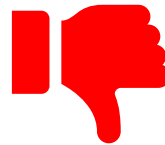
Es gibt in der Klasse Optional<T> folgende statische Factory-Methoden, um ein Optional-Objekt zu erzeugen:

1. `return Optional.empty();` *// ein leeres Optional-Objekt, quasi der Ersatz für null*
2. `return Optional.of(customer);` *// wirft NullPointerException wenn customer == null*
3. `return Optional.ofNullable(customer);` *// i. d. R. die Methode der Wahl.
Falls customer == null, return empty Optional, sonst ein Optional mit dem customer-Objekt.*

Optional<T> verwenden

Einige Methoden aus Optional<T>:

~~boolean isEmpty()~~ // *möglichst vermeiden*
~~boolean isPresent()~~ // *möglichst vermeiden*
~~T get()~~ // *möglichst vermeiden*



T orElse(T other)
T orElseGet(lambda)
T orElseThrow()
T orElseThrow(lambda)



void ifPresent(lambda)
void ifPresentOrElse(lambda, lambda)

Optional<U> map(lambda) // *Optional<T> kann .stream() und somit alle Stream-Methoden.* 🤖

Optional<T> or...



Man gewinnt mit den Optional-Methoden `isPresent` und `get` nichts – man hätte genauso gut `null` zurückgeben können statt ein `Optional`, weil man wieder eine `if`-Abfrage macht. Beispiel:



// null-Variante:

```
Customer customer = db.findCustomer(name);  
if (customer != null) { // null-Check  
    System.out.println(customer.name());  
} else {  
    System.out.println("NoName");  
}
```



// Optional-Variante mit isPresent und get:

```
Optional<Customer> customer = db.findCustomer(name);  
if (customer.isPresent()) { // quasi null-Check  
    System.out.println(customer.get().name());  
} else {  
    System.out.println("NoName");  
}
```



// orElse gibt entweder den Wert im Optional zurück, wenn dieser existiert; falls nicht, wird ein Default-Wert "NoName" zurückgegeben.
// Optional unterstützt auch die Stream-Methode map. Diese wird hier genutzt, um den name() zu extrahieren, sofern der customer existiert.
System.out.println(customer.map(Customer::name).orElse("NoName"));

Optional<T> statt **null**

Optional<T> sollte **als Rückgabewert** verwendet werden, falls das Objekt **null** sein könnte.

- Mit Optional kann **explizit in der Sprache ausgedrückt** werden, dass ein Objekt eventuell nicht vorhanden (**null**) sein könnte.
- Mit Optional<T> lässt sich bereits zur Kompilezeit **null** isolieren und damit NullPointerException vermeiden.

Hinweis: Die Verwendung von Optional<T> für Parameter ist umstritten.

Besser: Methoden-Overloading oder [Varargs...](#)

"API Note: Optional is primarily intended for use as a method return type where there is a clear need to represent "no result," and where using null is likely to cause errors. A variable whose type is Optional should never itself be null; it should always point to an Optional instance."

To optional or not to optional?



Variante 1: Optional

```
Optional<Invoice> calculateByCustomer(String name) {  
    Customer customer = database.findCustomer(name);  
    if(customer == null) {  
        // return Optional.empty();  
        // throw new NoSuchElementException("Customer " + name + " not found.")  
    }  
    double sum = customer.orders().stream().mapToDouble(order -> order.totalCost()).sum();  
    Invoice invoice = new Invoice(name, sum);  
    return invoice;  
}
```

Variante 2: Exception

Was tun, wenn kein customer gefunden wird?



Wann Exception verwenden?

- Exceptions sind in der Regel **langsam in Bezug auf Performanz**.
- Exceptions sind für **außergewöhnliche Situationen** gedacht.
Z. B. I/O-Probleme
- Exceptions werden auch für **Preconditions** verwendet.
- Verwende Exception, wenn es **keine alternative Handlung** gibt.
D. h., es lässt sich nichts weiter machen, außer den Fehler auszugeben.
- Verwende Exceptions **nicht**, um den **Kontrollfluss zu steuern**.
D. h., wenn sich etwas leicht mit if-else prüfen lässt, dann nutze if-else.

```
// do not do this:  
Student student = new Student("Sara");  
try {  
    return student.name(); // student can be null  
} catch (NullPointerException e) {  
    return "";  
}
```

```
// do this instead:  
if(student == null) {  
    return "";  
}  
return student.name();
```

```
// or even better use an Optional<Student>:  
return student.map(Student::name).orElse("");
```

Wann Optional<T> verwenden?

- Optional<T> sollte nur **für Rückgabewerte von Methoden** verwendet werden.
- **Ausnahme:** Wenn eine **Methode ein Iterable (Collection) zurückgibt**, ist es in der Regel besser, entweder **die Ergebnisliste** zurückzugeben oder **eine leere Liste** (z. B. bei Suchanfragen).
 - Ausnahme von der Ausnahme:
 - Eine Ergebnisliste darf nicht leer sein (z. B., um Division durch 0 zu vermeiden).
 - Es soll eine explizite Fehlermeldung an den User kommuniziert werden.
- Verwende Optional<T> **nicht für Parameter!**
Verwende stattdessen Method-Overloading oder Varargs.

Optional<T> Parameter vs Method Overloading

Optional-Parameter (Vermeide):

```
Optional<Customer> findCustomer(Optional<String> customerName) {  
    try {  
        String fileContent = Files.readString(Path.of(CUSTOMERS_DB_FILE));  
        return parseCustomer(customerName.orElse("defaultCustomer"), fileContent);  
    } catch (IOException e) {  
        throw new DatabaseException("File not found: " + CUSTOMERS_DB_FILE);  
    }  
}
```



Method Overloading (Best Practice):

```
Optional<Customer> findCustomer() { // Method-Overloading von findCustomer ohne customerName  
    return findCustomer("defaultCustomer"); // Verwende Implementation von findCustomer wieder. #DRY  
}
```

```
Optional<Customer> findCustomer(String customerName) {  
    try {  
        String fileContent = Files.readString(Path.of(CUSTOMERS_DB_FILE));  
        return parseCustomer(customerName, fileContent);  
    } catch (IOException e) {  
        throw new DatabaseException("File not found: " + CUSTOMERS_DB_FILE);  
    }  
}
```

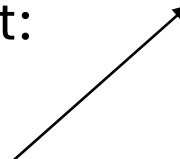


Optional<T> und Exception

Häufig werden Optional<T> und Exception auch kombiniert:

```
public Optional<Customer> findCustomer(String name) throws DatabaseException {  
    return findCustomer("defaultCustomer");  
}
```

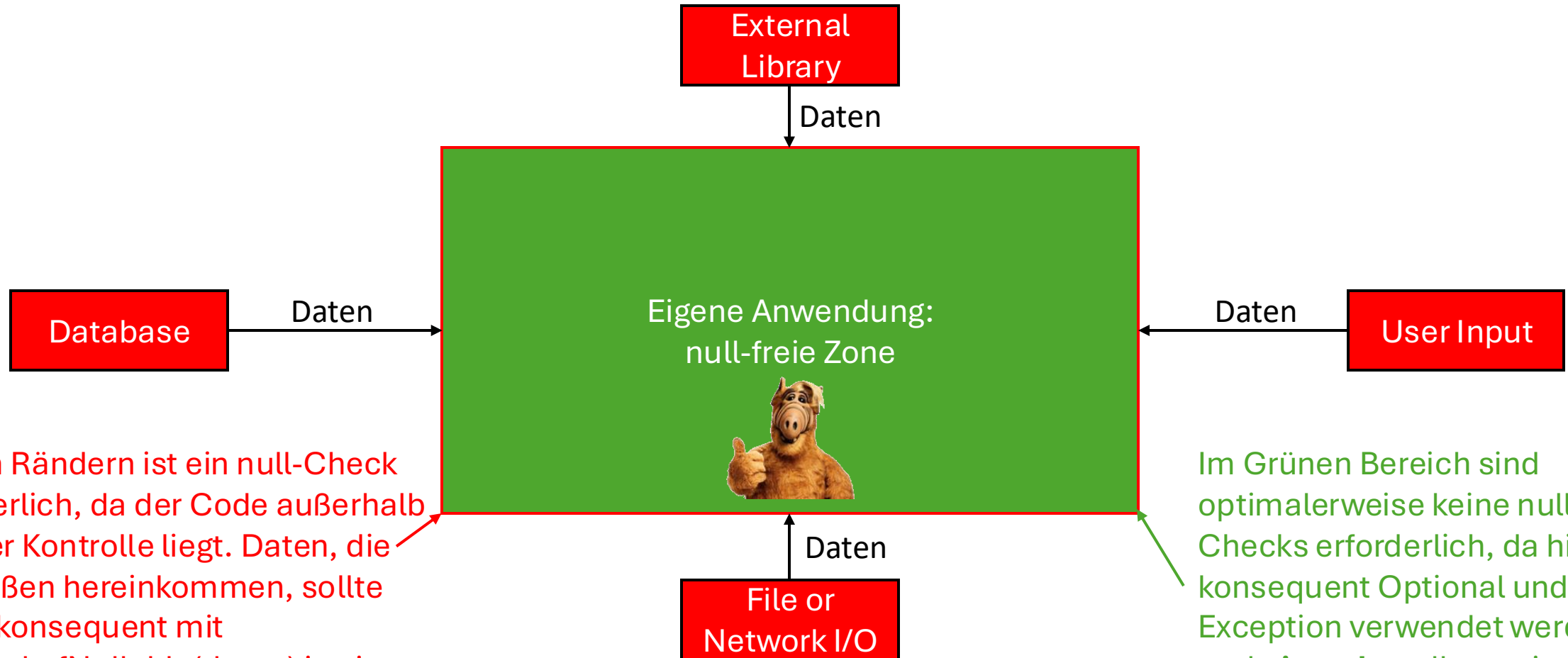
```
public class DatabaseException extends RuntimeException {  
    public DatabaseException(String message) {  
        super(message);  
    }  
}
```



Hier gibt findCustomer ein Optional<Customer> zurück, da eventuell kein Customer mit dem gesuchten Namen gefunden werden kann (was durchaus häufiger vorkommen kann und nichts Ungewöhnliches darstellt).

Außerdem wird eine unchecked DatabaseException geworfen, falls etwas bei der Verbindung zur Datenbank schief geht (was nicht vorkommen sollte, da dies eine weitere Nutzung der Anwendung stark einschränken würde).

To null-check or not to null-check?



An den Rändern ist ein null-Check erforderlich, da der Code außerhalb unserer Kontrolle liegt. Daten, die von Außen hereinkommen, sollte daher konsequent mit `Optional.ofNullable(daten)` in ein `Optional` umgewandelt werden.

Im Grünen Bereich sind optimalerweise keine null-Checks erforderlich, da hier konsequent `Optional` und `Exception` verwendet werden und **niemals** null von einer Methode zurückgegeben wird.

Lernziele Exceptions

- **Preconditions prüfen (Fail Fast):**

- Parameter auf Gültigkeit testen (z. B. `null-Prüfung`, `isEmpty()`).

- **Exceptions werfen:**

- **Unchecked:** `throw new IllegalArgumentException()` bei ungültigen Parametern.
- **Checked:** `throw new EigeneException()` bei erwarteten Fehlern.

- **Eigene Checked Exceptions:**

- Klasse erstellen, die von `Exception` erbt.
- In Methodensignatur mit `throws EigeneException` deklarieren.

- **Exceptions behandeln:**

- Risikocode mit `try-catch` absichern.
- **Umwandlung (Wrapping):** `catch (SomeException e) und throw new MyException(e).`

Lernziele Optional

- **Zweck:**

- Typsichere Alternative zu `null` *nur* für **Rückgabewerte**.

- **Optional erzeugen:**

- `Optional.empty()`: Für "kein Ergebnis".
- `Optional.of(value)`: Für "garantiertes Ergebnis".

- **Funktional verarbeiten (statt `isPresent()`):**

- `.orElse(defaultValue)`: Standardwert bei empty.
- `.map(lambda)`: Wert im Optional transformieren.

- **Streams & Optional:**

- Stream-Methoden wie `.findFirst()` oder `.average()` geben Optional zurück.

- **Wann NICHT verwenden:**

- Nie für Parameter.
- Nie für Collections (`Optional<Collection>`). **Besser:** Leere Collection zurückgeben.